ENERGYDRIVE SYSTEMS

Your first project. One click at a time.

A beginner-friendly walkthrough using GitHub and VS Code menus.

Prepare → Create & clone → Container & checks → Agent & first PR

Your own repository

An independent project under EnergyDrive-Systems, not a fork of the baseline.

A working environment

Containerized tooling, a synced Python .venv, and repeatable verification.

A prepared agent

Shared standards understood before discussing or implementing your application.

No terminal commands need to be typed in this walkthrough. Ask your project lead or IT contact for help. The template prepares development; it does not start a finished application.

Source: Your First Project guide

THE WORDS WILL BECOME FAMILIAR

A few words, in plain English.

Getting your workspace

- **Repository / repo:** your project's files and saved history on GitHub.
- **Template:** a prepared starting set of files.
- **Clone:** download a working copy linked to your GitHub project.
- **Container:** an isolated workspace with the tools you need.

Making reviewed changes

- **.venv:** the project's own Python environment, prepared for you.
- **Branch:** a separate workspace for changes before review.
- **Commit:** a named save point in your project's history.
- **Pull request / PR:** ask someone to review your branch before merging it.

ⓘ VS Code's **Command Palette** is a searchable menu, not a terminal. Selecting a menu action does not require learning a programming command.

ASK IT FOR INSTALLATION HELP

Get your computer ready.

01 Ask IT/project lead to install **Git, VS Code, and Docker Desktop** using approved processes.

02 Open **Docker Desktop** from the Start menu. Wait until its engine is running.

03 Open VS Code > **Extensions** (four-squares icon). Search **Dev Containers** by Microsoft and click Install.

04 Use VS Code's **Accounts** menu and your browser to sign in to GitHub when needed.

You should see

Docker is running and Dev Containers is installed. Git works behind VS Code's buttons.

Host Python is not required. Internet is needed for setup; agent accounts/subscriptions are separate.

Windows uses Linux containers and may need WSL 2. IT handles that setup, licensing, and administrator changes.

Mac: Command instead of Ctrl. Linux: ask your helper about folder permissions.

THE ORGANIZATION OWNS THE NEW PROJECT

Create, don't fork.

01 Open EnergyDrive-Systems/BaselineRepo.

02 Choose **Use this template > Create a new repository**, not Fork.

03 Select owner **EnergyDrive-Systems**, not your personal account. Enter a name such as **my-first-project**.

04 Choose **Private** unless public use is approved. Leave all-branches inclusion off.

05 Create and check the organization and project name on the page.

You should see

EnergyDrive-Systems / my-first-project on GitHub, with README and the .devcontainer folder.

If your lead already created it, use their project link and continue; don't create a duplicate.

No template button or organization owner option? Ask your lead. Do not publish personally as a workaround.

Your project lead configures owners, protections, and deployment settings. You do not need to administer them just to open the project; they are not automatically copied with files.

Use your actual agreed project name; my-first-project is only an example.

CLONE YOUR NEW PROJECT, NOT BASELINEREPO

Clone and open VS Code.

01 On your **new project's** GitHub page, click **Code > HTTPS > copy icon**.

02 In VS Code choose **View > Command Palette** (Ctrl+Shift+P). Search **Git: Clone** and select it.

03 Paste the address, press Enter, and complete browser sign-in if asked.

04 Choose a parent folder through the folder picker, for example C:\repos. Create it in File Explorer if needed.

05 When finished, click **Open** or **Open in New Window**.

You should see

The right project name and README in **Explorer** (files icon), with no changed files in Source Control initially.

VS Code creates the project folder inside the selected parent folder. Don't reuse an existing folder with the same name.

Ask your helper if the project connection is unclear. Clone the new project, not BaselineRepo.

Authenticate through the approved GitHub browser/credential flow. Never put passwords or tokens in commands, docs, remote URLs, or agent chats.

CHOOSE THE EXECUTION ENVIRONMENT EXPLICITLY

Reopen in Container.

01 Review the project setup and trust the workspace if appropriate.

02 Click **Reopen in Container**. If no prompt appears, search **Dev Containers: Reopen in Container** in the Command Palette.

03 Wait for preparation to finish. First startup can take several minutes.

04 Use the reopened window for project work. You do not need to type into a terminal.

You should see

The bottom-left connection indicator identifies a **Dev Container**.

The project's Python environment and dependencies are prepared automatically. Python, Pylance, and Ruff install inside it; accept prompts if shown.

If the window is still local or an error appears, stop and ask for help before editing.

Opening a folder alone does not silently attach or launch an app. No database, gateway, Terraform operation, app server, or production connection starts here.

SELECT AN ACTION, DON'T TYPE A COMMAND

Run checks from a menu.

01 In the reopened window choose **Terminal > Run Task**.

02 If your menu differs, search **Tasks: Run Task** in the Command Palette.

03 Select **Baseline: Run All Quality Checks**.

04 Let the results panel finish. This panel shows output; you don't need to type into it.

You should see

Six checks run: environment, code quality, formatting, types, tests, and dependency audit.

Each finishes with **PASS**. The summary says:

All six quality checks passed.
Your development setup is ready.

The audit needs internet access. These tasks check files; they do not fix or deploy them.

⚠ A check failure is something to report, not hide. Tell your helper which check failed and show its output; do not disable checks or change versions to make it disappear.

The tasks require the prepared container and refuse the wrong execution location.

YOU DON'T NEED TO INTERPRET EVERY LOG LINE

Know what success looks like.

Expected results

- **Environment ready**, Python 3.13.x, and /workspace/.venv.
- Code quality, formatting, and type checks pass; types show zero errors.
- Tests pass; the starter currently has 12.
- No unresolved audit findings. A missing src message is normal before app code if types still show zero errors.

A visual test-results view

01 Click **Testing** (the beaker icon), then **Run All Tests**.

02 If hidden: **View > Open View > Testing**.

03 If tests don't appear: **Python: Configure Tests > pytest > tests folder**, or ask for help.

04 The menu task **Baseline: Run Tests** remains an alternative.

If Python is wrong, select **Python: Select Interpreter** and choose /workspace/.venv/bin/python. You don't need to locate a Windows .venv folder; ask your helper if unsure.

INSTALL AND SIGN IN INSIDE THE RIGHT WORKSPACE

Start your agent.

Claude Code in VS Code

01 Stay in the **container-connected** window.

02 In Extensions, install **Claude Code by Anthropic** (`anthropic.claude-code`) in the Dev Container when offered.

03 Open its panel from the Command Palette and complete the approved interactive sign-in.

04 Choose a mode that lets you review edits/commands. Do not enable permission bypass for this test.

The chat panel is enough

Claude is **not installed automatically**. Install its extension while connected to the container; the panel is sufficient for this guide.

You do not need a terminal-based Claude installation. Ask your technical lead if your team requires one later.

For Copilot/Codex, install and sign in to the approved client. Ask it to prove that its tools use the container, not your computer's Python.

Accounts, availability, and skill discovery vary by client. Use normal review/permission prompts while learning.

Subscriptions/accounts are not supplied. Complete authentication directly in the UI; never send credentials to the agent.

FIRST SESSION: UNDERSTAND, DON'T EDIT

Send this read-only prompt.

```
I am new to this project. Read the repository's shared agent instructions,
    README, standards, handover, and skill index. Do not change any files,
    commit, push, deploy, or connect to external databases or equipment.
    Explain the main rules in plain language, including Material 3 UI design,
    review, bug-fix evidence, documentation, and production restrictions.
    Use the project's environment check inside the development container.
    Show the actual result and tell me whether setup is ready. If a tool is
    unavailable, say so rather than guessing. Wait for me before making changes.
```

✓✓ You should see

The agent explains the rules, shows a real environment result with **/workspace** and **/workspace/.venv**, and makes no edits.

If a skill is unavailable

Ask the agent to read the named **SKILL.md** file directly. You do not need to install a large plugin collection to start.

A confident policy summary is not proof of tool location; require actual command output.

START WITH THE GOAL, NOT THE CODE

Describe what you want to build.

```
I want to build [describe the goal] for [the users]. Use the requirements
    interview and domain-modeling workflows. Ask me the necessary questions
    and recommend sensible choices. Do not implement anything until we agree.
    Help me document the requirements and current project state afterward.
```

Confirm the plan

Describe the purpose, users, and what success looks like. Answer the agent's questions and approve its summary before allowing implementation.

New/changed UIs use **Material Design 3** by default.

Where to see it

In Explorer, open **docs > PRD.md** for requirements and **HANDOVER.md** for current state/next steps.

Ask the agent to replace template-history entries with your project's state, preserving shared policy.

KEEP CHANGES SEPARATE UNTIL REVIEWED

Create a branch with a menu.

01 Click the branch name, usually **main**, in VS Code's bottom-left status bar.

02 Choose **Create new branch**. If needed, search **Git: Create Branch** in the Command Palette.

03 Name it **setup/project-bootstrap**.

04 Confirm that name is visible before allowing the agent to edit project files.

You should see

The status bar shows **setup/project-bootstrap**, not main.

Ask the agent to update PRD, handover, and **docs/changes/project-bootstrap.md**.

The branch keeps your proposed work separate for review; it does not create a second GitHub repository.

⚠ If you are still on **main**, stop before committing or publishing changes. Ask your helper if the branch menu is unclear.

PROJECT SETUP IS A REVIEWED CHANGE

Open your first setup PR.

In VS Code

01 Open **Source Control** (branch icon). Click changed files to review the differences.

02 Stage intended files with their + buttons; don't include secrets/generated data.

03 Enter a message such as **Set up project requirements** and click **Commit**.

04 Click **Publish Branch**. Ask for help with identity/sign-in dialogs; never share credentials in chat.

On your project's GitHub page

01 Click **Compare & pull request**, or **Pull requests > New pull request**.

02 Set base to **main** and compare to **setup/project-bootstrap**.

03 Add a description/evidence, create the PR, and request the designated reviewer.

04 Wait for passing required checks and independent approval before merge.

⚠ Use **Publish Branch**, not Publish to GitHub to create another repository. Don't publish directly on main or bypass review/checks.

KEEP YOUR WORK AND ENVIRONMENT

Come back to the project later.

01 Save files and close the project window.

02 Next time, start **Docker Desktop**.

03 Use **VS Code > File > Open Recent** and select the project.

04 Choose **Reopen in Container** if needed.

05 Check the connection indicator and rerun **Baseline: Run All Quality Checks**.

Your ready-to-start checks

- Correct organization project and folder.
- Dev Container connected; six checks pass.
- Agent reads rules, shows actual results, and waits for approval.
- Requirements agreed before implementation.
- Changes use a reviewed branch/PR.
- Reopening restores passing checks.

Don't delete containers/volumes just to close the window. They may hold development data. Ask your helper before reset or deletion operations.

STOP AT THE FIRST UNEXPLAINED ERROR

Check the screen, ask for help.

Wrong owner / can't create	Ask your lead to create the organization project or arrange transfer. Don't use a personal repo as a workaround.
Repository not found	Check the address copied from your new project and browser sign-in; ask your lead about access.
No template button / missing files	Ask the template owner to publish the current version and enable template use.
Git missing / no reopen or tasks	Ask IT to install Git and restart VS Code. Check Microsoft's Dev Containers extension and the copied template version.

No terminal commands needed; tell your helper which step/button failed.

DON'T HIDE OR WORK AROUND A FAILURE

Show the error, not your secrets.

Environment and checks

- **Docker/setup fails:** open Docker Desktop, wait for its engine, and share the first Dev Containers log error with IT.
- **Check task / Python wrong:** confirm the Dev Container indicator; use Python: Select Interpreter or ask for help.
- **No tests in Testing:** use Baseline: Run Tests from Run Task; ask for discovery help.

Agents and saving work

- **Claude missing/not signed in:** install in the container window and complete browser sign-in.
- **Commit/Publish Branch unavailable:** check your branch/Source Control and ask about identity/sign-in. Don't create another personal repo.
- **Permission/lockfile error:** ask your lead; don't switch to root, edit locks, or weaken checks.

Share the step, what you clicked, expected result, and error. A redacted screenshot helps. Find versions in VS Code **Help > About** and Docker's About menu. Never share sign-in codes or tokens.

NOT A BEGINNER PREREQUISITE

What your owner/helper handles.

Publish a ready template

- Enable Template repository and merge intended changes through a reviewed PR.
- Verify the current guide, .devcontainer setup, **.vscode/tasks.json**, Compose, locks, and agent instructions are published.
- Copying GitHub never includes local, uncommitted changes.
- Configure actual CODEOWNERS, protections, environments, and approvals per project.

Resolve technical prerequisites

- Handle installation/licensing, Linux folder access for UID 10001, and optional Terraform/cloud tooling.
- Do not mount Docker/production credentials merely to make beginner checks work.
- Change dependencies/metadata intentionally through uv, then verify; never hand-edit locks.
- The written guide's helper appendix contains optional technical diagnostics and official Claude setup references.

Passing setup checks is not proof of every agent's skill discovery, remote CI, real database/gateway behavior, accessibility conformance, or production readiness.